PYTHON MODULE 04

Sure! Here's a detailed but simple and exam-ready explanation of **Errors and Exceptions** in Python, covering all the subtopics with clear headings and complete sentences:

🌟 Errors and Exceptions in Python

📌 Introduction to Errors and Exceptions

In Python, when something goes wrong during the execution of a program, it is called an **error**. Errors stop the program unless they are handled. There are two main types:

1. **Syntax Errors** – These occur when the code is not written correctly. Example: missing a colon or bracket.
2. **Exceptions** – These occur during the program's execution. The code is correct, but something unexpected happens (like dividing by zero or accessing a file that doesn't exist).

✅ Difference between Errors and Exceptions

Errors	Exceptions
Happen due to wrong syntax.	Happen due to unexpected events.
Cannot be handled easily.	Can be handled using try-except.
Occur before code runs.	Occur while code is running.

🔧 Handling Exceptions

Python provides a way to handle exceptions using the try, except block.

Syntax:

```
try:
    # code that may raise exception
except SomeException:
    # code to handle the exception
```

Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

This avoids program crashing and shows a friendly message.

□ Multiple Exceptions

Sometimes, we want to handle different exceptions separately. We can do this using multiple except blocks.

Example:

```
try:
    x = int("abc")
except ValueError:
```

```
    print("Value Error occurred.")
except ZeroDivisionError:
    print("Division by zero.")
```

Each except handles a specific type of exception.

📣 Raising Exceptions

You can manually raise exceptions in your code using the raise keyword.

Syntax:

```
raise ExceptionType("Custom message")
```

Example:

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative.")
```

This is useful for creating meaningful error messages and stopping invalid operations.

🔗 Exception Chaining

Sometimes, an exception is raised while another exception is already being handled. In this case, you can **chain exceptions** using the from keyword.

Syntax:

```
try:
    raise ValueError("Original error")
except ValueError as e:
    raise TypeError("New error occurred") from e
```

This helps in debugging by showing the full chain of errors.

🔤 Built-in Exceptions

Python has many built-in exceptions. Some common ones include:

Exception Name	Description
ZeroDivisionError	Dividing by zero
ValueError	Invalid value (e.g., int("abc"))
TypeError	Wrong data type
IndexError	Index out of range
KeyError	Missing key in a dictionary
FileNotFoundError	File doesn't exist

Each exception is a class and has a message that explains the error.

🦸 User-Defined Exceptions

You can create your own exceptions by making a class that inherits from Exception.

Syntax:

```
class MyError(Exception):
    pass


raise MyError("This is a custom error.")
```

Why use it?
To make your program more readable and handle specific problems in your application.

□ Clean-Up Actions

Sometimes you need to clean up resources (like closing files) whether an exception happens or not. For this, Python uses the finally block.

Syntax:

```
try:
    # risky code
except:
    # handle error
finally:
    # cleanup code (runs no matter what)
```

Example:

```
try:
    f = open("file.txt")
    data = f.read()
except FileNotFoundError:
    print("File not found.")
finally:
    f.close()
```

The finally block always runs and is useful for cleanup actions like closing files or releasing resources.

🎯 Summary

- **Errors** are syntax mistakes; **exceptions** are problems during execution.
- Python uses try-except to handle exceptions.
- You can handle **multiple exceptions** and even **raise your own**.
- Python has many **built-in exceptions**, and you can create **custom exceptions**.
- Use finally for **clean-up**, no matter what happens.

Let me know if you'd like a PDF version of this or practice questions on this topic!